GridPact: agent usability and final project slides

Homework 9, MAS.665 AI Studio, Fall 2026. Team: Nicolas Hong, Maheen Rassell.

Links	
Interface documentation	https://github.com/nh13120/gridpact/blob/hw9-agent-usability/docs/AGENT_GUIDE.md
Our slides in the shared Demo Day deck	[paste the link to our slides in the shared Demo Day deck here]
Code (GridPact)	github.com/nh13120/gridpact
HW8 pull request (Nanda Town)	github.com/projnanda/nandatown/pull/345
Cold-start plan, logs, scorer	`coldstart/` in the repository: `PLAN.md` (committed before the runs), `runs/run1..4/` (proxy logs, database, scores)

Contributions

Person	Contribution
Nicolas Hong	GridPact itself: the protocol, ledger, MCP server and Haiku agents; the HW7 preregistration, benchmark harness, runs and the v2 change that this work builds on.
Maheen Rassell	HW6 audit fixes (bounty accounting, money-conservation check, settlement tests); HW7 write-up; HW8 Nanda Town integration and pull request; HW9 agent guide, cold-start test bed and scorer, the get_commitments improvement and tests, final slides and this report. The HW8 and HW9 engineering was done with an AI coding assistant (Claude Code), which also ran the cold-start test bed described below.

1. The interface

GridPact is shared infrastructure for energy agents: a marketplace and referee. Agents trade energy for windows of 15-minute steps, a seller locks a deposit (stake) against its promise, plain code settles delivery, and an auditor can challenge a false capacity claim. It is not an agent itself. The full guide, written for an agent that has seen nothing else, is docs/AGENT_GUIDE.md. Summary:

Access	MCP over streamable HTTP, e.g. `http://127.0.0.1:8765/mcp`. The guide gives the three-request HTTP handshake (initialize, notifications/initialized, tools/call) and a working curl example, so any client that can POST JSON can use it.
Tools	register_agent, find_counterparties, propose, respond, get_inbox, get_commitments (added in this homework), get_ledger, challenge. Every reply is JSON with `ok` and, on refusal, a `reason`.
Inputs and outputs	Time is 96 steps of 15 minutes (step 68 = 17:00); energy in kWh per step, prices in $/MWh. The guide tabulates every tool's inputs and returned fields.
Permissions and limits	Roles buyer, seller, auditor; a secret token from registration (shown once) is the identity. Caps on kWh per step; the seller's stake is locked at acceptance; only auditors challenge; a valid challenge pays 10% of the stake, a wrong one costs a $50 bond; one valid challenge per commitment. Agents cannot settle: the operator closes the day.
Setup	`GRIDPACT_DATA_DIR=fixtures/synthetic python -m gridpact.server` (made-up registry, no data files or API key needed).
One small example	COM_09 registers, proposes 500 kWh/step to PVB_17 for steps 68 to 79 at $50/MWh with a $60 penalty (request_id 'order-1'); PVB_17 reads get_inbox and accepts; the commitment locks a $360 stake (60 x 500 x 12 / 1000).

2. Cold-start test

Question. Can an agent that has only the guide, a URL and a task use GridPact for a real job? **Setup.** Three fresh Claude sessions ran at the same time as a buyer (COM_09), an honest seller (PVB_17) and an auditor (AUD_1), each with its own logging proxy in front of one GridPact server on the synthetic registry. Each was given the guide, its URL and a one-paragraph task. No call sequence and no expected answer were given; the author passed no messages between them. They coordinated only through GridPact. The auditor was also authorised to read the public registry file, because the nameplate is only there. A seller PVB_37, which declares 5,000 kW for a 2,500 kW battery, was

registered by its owner as part of the setup, with a small script that accepts any offer made to it.

Success defined before the run. `coldstart/PLAN.md` (commit 74055c8, before run 1) fixes the task texts and 15 checks (a 16th, B8, was added before run 4), scored from the database and the proxy logs rather than from what a client says:

#	Check	Run 1	Run 2	Run 3	Run 4
B1	buyer registered as COM_09	PASS	PASS	PASS	PASS
B2	a commitment covers steps 68 to 79	PASS	PASS	PASS	PASS
B3	at least 1,500 kWh/step committed	**FAIL**	**FAIL**	PASS	PASS
B4	all prices at or below $55	PASS	PASS	PASS	PASS
B5	no double-booking, no cap exceeded	PASS	PASS	PASS	PASS
B6	PVB_99 request refused, nothing created, reported	PASS	PASS	PASS	PASS
B8	step-range limit refused live on a registered seller (added for run 4)	not scored	not scored	not scored	PASS
B7	buyer's reported totals match the database	**FAIL**	**FAIL**	PASS	PASS
S1	PVB_17 registered as seller	PASS	PASS	PASS	PASS
S2	every offer to PVB_17 answered	PASS	PASS	PASS	PASS
S3	price floor ($45) and cap respected	PASS	PASS	PASS	PASS
A1	an auditor registered	PASS	PASS	PASS	PASS
A2	valid challenge on PVB_37 before settlement	PASS	PASS	PASS	PASS
A3	no wrong challenge (honest plant not accused)	PASS	PASS	PASS	PASS
Y1	ledger hash chain intact	PASS	PASS	PASS	PASS
Y2	liar's commitment cut to its real cap; buyer not out of pocket	PASS	PASS	PASS	PASS
	Checks passed	**13 of 15**	**13 of 15**	**15 of 15**	**16 of 16**

What the clients did, and how they used the result

In run 1 the buyer registered, listed sellers, proposed 1,250 kWh/step to PVB_37 and 250 to PVB_17 at $50/MWh, polled its inbox until both were accepted (about a minute), and reported 18,000 kWh for $900. The seller polled for the full ten minutes and accepted the one offer that fit. The auditor wrote its own HTTP client from the guide, found PVB_37's declared 5,000 kW in the ledger, compared it with the registry nameplate (2,500 kW), challenged, and was ruled valid ($90 bounty); it left the honest PVB_17 alone. The server cut PVB_37's commitment to 625 kWh/step and released $450 of its stake. After settlement the ledger chain verified and the liar's delivery matched its real battery.

Where the result use failed (run 1). The auditor's challenge cut the buyer's commitment at 16 s. The buyer next polled at 42 s, but `get_inbox` shows only that an offer is 'accepted', not its current volume. It reported 18,000 kWh and $900; the database held 10,500 kWh and $525 (B3 and B7). **Classification: interface limitation.** The buyer followed the guide, nothing was refused, and no tool could have told it. It is not a test-setup problem: the proxy, the server and the liar's script behaved as designed, and the cut is GridPact working as intended.

Other friction the clients reported (all recorded in the fixes below): no stated way to read the registry parquet file (system Python had no parquet reader; the auditor borrowed the repository's virtual-environment interpreter, which we disclose as a deviation from 'documentation only'); the guide did not list the ledger's action names, the inbox's fields, that steps are inclusive, or that a seller must register before it can be proposed to. The shell's 120-second background rule and a missing `timeout` command on macOS were environment issues, not GridPact's.

3. Boundary case, improvement, rerun

Boundary tested. Every run asked for PVB_99 (steps 100 to 110). GridPact answered `{"ok": false, "reason": "PVB_99 is not registered"}`, created no offer, and every client reported it accurately (B6). Because that refusal comes before the step range is checked, runs 1 to 3 never reached the range limit. Run 4's buyer was also told to ask a registered seller for steps 100 to 110 and got `steps must satisfy 0 <= step_from <= step_to < 96`, nothing created (B8). `tests/test_boundaries.py` covers these and the other limits directly (reversed steps, over-cap volume, negative price, empty request_id, wrong role, wrong token, a second challenge on one commitment): each is refused with a

reason and changes no offers, commitments or ledger entries.

Improvements, in order. (1) After run 1, a read tool `get_commitments` (current volume, original volume if cut, price, stake, status) and guide fixes for the friction the clients listed. (2) After run 2, **notices**: when a valid challenge cuts a commitment, both parties are told on their next successful call, on any tool, once. Regression tests: `tests/test_get_commitments.py` and `tests/test_notices.py` (40 tests pass in all).

Rerun results. Run 2 (13 of 15): the tool worked, but the buyer called it once at 46 s, before the challenge at 107 s, then polled only `find_counterparties` until 352 s and reported a stale position (18,000 kWh, $990 against 10,500 kWh and $578 in the database). Run 2 is kept as it happened. Its cause was an interface that pushes nothing, which a guide note could not fix, so we added notices. **Run 3** (15 of 15, guide note only): the buyer re-read at 557 s, found the cut, bought the missing 625 kWh/step and reported correctly; it polled for 580 s against run 2's 352 s, so the runs are not matched. **Run 4** (16 of 16, notices, fresh clients, same tasks plus the range probe): the auditor's valid challenge cut PVB_37 from 750 to 625 kWh/step; the buyer's next call returned a notice saying so, it bought the missing 125 kWh/step from PVB_17, re-read `get_commitments`, and reported 18,000 kWh and $990, matching the database. The same failure mode as run 2 (a buyer that keeps making other calls) now ends correctly.

What improved: an agent can see a cut and is told about it on whatever call it makes next; the guide covers the friction the clients listed; the blind spot of runs 1 and 2 has regression tests and a passing rerun. **What did not:** a buyer that makes no further call still holds a stale position; the nameplate is still only in a file that needs a parquet reader; the seller learns only by polling. **Unproven:** the clients are fresh Claude sessions started by the author, not another builder's agents, so this is a **self-run** test with no independent client; there is one run per condition and the auditor's timing varies between runs, so we claim no success rate; only buyer, seller and auditor roles and one lie were tested.

4. Repeat the test

```
# 1. server, logging proxies and the liar's owner script; prints three client URLs
python -m coldstart.run_env --out coldstart/runs/mine
# 2. give each fresh client docs/AGENT_GUIDE.md, its URL and its task from coldstart/PLAN.md
# 3. score the 15 checks (--settle closes the day as the operator)
python -m coldstart.check coldstart/runs/mine --reports reports.json --settle
# unit, boundary and regression tests (40)
pytest
```

Disclosure: self-run; the clients were fresh Claude subagent sessions of the same model family as the author's assistant; manual help during the runs: none; the author wrote the guide, the test bed and the checks; the clients shared a working directory with the repository and were told to read only the guide (their reports list the files they read; the auditor also used the repository's Python interpreter, noted above). In run 2 the seller's polling ended at about 560 of 600 seconds because the test bed was stopped to start run 3; no offer was pending, so no outcome changed. Run 4's test bed was stopped when scoring began, after every offer to the seller had been answered.

Final project slides

Export of our slides. Live deck: [paste the link to our slides in the shared Demo Day deck here]. The embedded demo video is checked in the live deck.

GRIDPACT 1 / 4

THE PROBLEM

AI agents will trade grid power.

Who checks their promises?

Buyers
need energy

Sellers
promise it

Auditors
check the promise

NICOLAS HONG · MAHEEN RASSELL PR GITHUB.COM/PROJNANDA/NANDATOWN/PULL/345

GRIDPACT 2 / 4

WHAT WE BUILT

A referee for agents.

01 Deposit
A seller locks money behind its promise.

02 Check
An auditor catches false claims.

03 Settle
Plain code pays the buyer. No AI.

30 SECOND VIDEO

Any agent connects over MCP from one written guide.

NICOLAS HONG · MAHEEN RASSELL PR GITHUB.COM/PROJNANDA/NANDATOWN/PULL/345

THE EVIDENCE

Tested. Then repaired.

50/50
liars caught

90%
of the need delivered, up from 65%

1,520
agents in 6 seconds

16/16
checks passed by a fresh agent, from our guide only

Our tests found two real gaps. We fixed both and reran.

WORKED · FAILED · NEXT

What we learned.

Worked	Failed	Next
Lying loses money.	Missed a second liar at scale.	A second audit pass.
Pre-registered, rerun passed.	Delivery is estimated, not metered.	Live meter data.
A fresh agent used it.	Only one lie tested.	Pilot with one ENGIE community.